

Project 702: Repo Review and Required Changes

*Review of `openchoreo/openchoreo` and `openchoreo/backstage-plugins` (main branch, shallow clones read on 9 Oct 2026). Everything marked **Verified** was read in source. Items marked **[Inference]** are my design judgement. I did not build or run either repo.*

1. Bottom line

1. **The core of Project 702 lives almost entirely in `backstage-plugins`.** For the MVP (GitHub + Flux + ReleaseBinding promotion) I found no required change in `openchoreo/openchoreo`. The proposal's non-goal "no controller changes" holds.
2. **No GitOps-aware code exists in `backstage-plugins`.** A search for "gitops" across its plugins, packages and configs returns nothing. Every portal write goes straight to the control-plane API. There is also no GitOps-mode flag to hook into.
3. **`occ` already does most of the rendering you need, offline.** `occ componentrelease generate` and `occ releasebinding generate` exist and run in file-system mode against a repo checkout. But they are not the commands in your slide deck, and they work differently from what your proposal assumes (see section 2).
4. **Four things must be built:** (a) GitOps scaffolder actions, (b) a way to ship and run `occ` inside the Backstage backend, (c) a GitOps switch in the Deploy tab so Promote and Overrides open a PR instead of writing to the API, and (d) a change tracker that powers the status chip.
5. **Several assumptions in your current files do not match the code.** Fix these before building (section 2).

2. Corrections to the current proposal and template

Your current assumption	What the code shows	Impact
Slide deck: <code>occ generate component --name ... --env ... --image ...</code>	No such command. Real commands: <code>occ componentrelease generate</code> (flags: <code>--all / --project / --component, --name, --namespace, --output-path, --dry-run, --mode, --root-dir</code>) and <code>occ releasebinding generate</code> (adds <code>--target-env, --use-pipeline, --component-release</code>). Verified in <code>internal/occ/cmd/*/cmd.go</code> .	Update the deck and the render action design.
<code>occ</code> can render in any context	<code>releasebinding generate</code> refuses anything except <code>--mode file-system</code> ("only supports file-system mode"). It indexes the YAML files in a repo checkout (Components, ComponentTypes, Traits, pipelines) from <code>--root-dir</code> .	The render step needs a full clone of the GitOps repo in the scaffolder workspace first. This is your "dependency gap", now confirmed.
Layout is a pattern like <code>namespaces/{namespace}/projects/{project}/...</code>	<code>occ</code> already has configurable layout, but it is a <code>release-config.yaml</code> at the repo root with <code>defaultOutputDir</code> plus per-project and per-component directory maps (<code>internal/occ/fsmode/config/release-config.go</code>). There are no <code>{placeholder}</code> patterns.	Decide: use the existing file format, or add pattern support as an upstream PR to <code>openchoreo/openchoreo</code> .
<code>openchoreo:gitops:validate, resolve-path, check-conflicts, render</code>	None exist. The scaffolder module registers 16 actions, all API-write actions (project, component, resource, environment, namespace, pipeline, types, traits, workflows).	These four actions are new work. <code>validate</code> and <code>resolve-path</code> can probably merge into one.
<code>occ</code> is available where Backstage runs	<code>occ</code> is not in the Backstage image. The only <code>occ</code> Dockerfile is <code>cmd/occ/Dockerfile</code> in the platform repo.	Needs a packaging decision (section 4.2).
Overrides can be rendered like promotion	<code>releasebinding generate</code> updates only <code>releaseName</code> on an existing binding and preserves other fields (it reads the original file, <code>IsUpdate</code> mode). <code>setoverride</code> (the <code>--set</code> engine) is only wired to API-mode <code>component, project</code> and <code>workflow</code> commands.	Override changes need a separate YAML patch step (for example Roadie's <code>jsonata:yaml:transform</code> , or a small TypeScript patcher). Not in <code>occ</code> file mode today.
Environment enum <code>development/staging/production</code> in <code>scaffolder.yaml</code>	Environments and promotion paths come from the DeploymentPipeline (the Deploy tab reads <code>promotionTargets</code> per environment).	Replace the hard-coded enum with a picker driven by the pipeline. <code>DeploymentPipelinePicker</code> and <code>NamespaceEntityPicker</code> already exist.
<code>publish:github:pull-request</code> must be added	Already installed. <code>packages/portal-backend</code> depends on <code>@backstage/plugin-scaffolder-backend-module-github</code> and loads it in <code>features.ts</code> .	One less dependency.

Your current assumption	What the code shows	Impact
Deterministic branch plus <code>update: true</code> is purely a convenience	It means a second promotion of the same component and environment silently updates the same PR .	Your conflict gate must decide what "overlap" means, because the template itself already reuses one branch per component and environment.
Deploy tab is a row of environment cards (my earlier doc)	Judging by file names, it also has a pipeline graph (<code>PipelineDAG/PipelineCanvas.tsx</code> , <code>DeployFlowCanvas.tsx</code> , <code>MiniEnvironmentNode</code>), an <code>EnvironmentDetailPanel</code> , and release dialogs (create release, release browser, release diff). I have not rendered it.	Verify visually before designing UI changes. My first doc understated this.

3. What exists today in openchoreo/backstage-plugins (Verified)

3.1 Plugin map

Package	Relevance to Project 702
<code>plugins/openchoreo</code>	Frontend core. Deploy tab, overrides page, scaffolder form fields, token decorator, NFS extensions in <code>src/alpha.tsx</code> . Main change area.
<code>plugins/openchoreo-backend</code>	Backstage backend plugin. Proxies the portal to the OpenChoreo API (<code>src/router.ts</code> , <code>src/services/EnvironmentService/EnvironmentInfoService.ts</code>).
<code>plugins/scaffolder-backend-module-openchoreo</code>	Registers OpenChoreo scaffolder actions (<code>src/module.ts</code> , <code>src/actions/*.ts</code>). Place for new GitOps actions (or a sibling module).
<code>plugins/openchoreo-common</code>	Shared types, including feature flags (<code>src/types/features.ts</code>).
<code>plugins/catalog-backend-module-openchoreo</code>	Catalog sync, exports <code>immediateCatalog</code> and <code>annotationStore</code> used by actions.
<code>packages/portal-backend</code> , <code>packages/portal-app</code>	The shipped portal composition. Backend already loads the GitHub scaffolder module and the GitHub auth provider.

Package	Relevance to Project 702
<code>templates/</code>	14 <code>create-openchoreo-*</code> templates (platform resources). No deploy or promote template.
<code>plugins/openchoreo-ci</code> , <code>-observability</code> , <code>-workflows</code>	Not needed for the MVP.

3.2 Every portal write that bypasses Git today

The frontend client (`plugins/openchoreo/src/api/OpenChoreoClient.ts` , interface in `OpenChoreoClientApi.ts`) calls these. Each one is a direct API write that a GitOps project would have to redirect or block.

Client method	UI trigger	Backend route / effect
<code>promoteToEnvironment</code>	Promote button	<code>POST /promote-deployment</code> , which calls <code>.../components/{c}/promote</code> on the API
<code>patchReleaseBindingOverrides</code> , <code>updateReleaseBinding</code>	Overrides page save	<code>PATCH / PUT</code> release binding
<code>deployRelease</code>	Deploy a release to the first environment	<code>POST</code> deploy release
<code>createComponentRelease</code>	Create release dialog	<code>POST</code> create release
<code>updateComponentBinding</code>	Undeploy / Redeploy	<code>PATCH</code> binding <code>releaseState</code>
<code>rolloutRestartReleaseBinding</code>	Rollout restart	<code>POST</code> (stamps <code>openchoreo.dev/restartedAt</code>)
<code>deleteReleaseBinding</code>	Remove deployment	<code>DELETE</code> binding
<code>patchComponent</code>	Auto-deploy toggle	<code>PATCH</code> component

Client method	UI trigger	Backend route / effect
<code>applyWorkload</code> , <code>updateComponentTraits</code> , <code>updateComponentConfig</code>	Workload, trait and config edits	direct writes

For the MVP, only the first two (promote, overrides) need a PR path. The rest need a **block or warn** behaviour in GitOps mode, otherwise Flux reverts them on the next sync (your "mixed mode" problem).

3.3 Seams for the Deploy tab change

- `Environments/hooks/useEnvironmentActions.ts` has `handlePromote(source, target)` , which calls `client.promoteToEnvironment` and then refetches. **This is the single place to branch on GitOps mode for promotion.**
- `Environments/hooks/usePromotionAction.ts` and `components/PromotePrimaryAction.tsx` and `EnvironmentActions.tsx` decide button label, disabled state and in-flight state. The in-flight tracker is keyed on environment name and assumes the promotion finishes quickly. A PR flow is **asynchronous and long-lived**, so "in flight" must become "PR open".
- `Environments/EnvironmentOverridesPage.tsx` (about 1,060 lines) plus `hooks/useOverrideChanges.ts` handle override edits and save.
- Permissions are checked with ABAC hooks (`useReleaseBindingUpdatePermission` , `usePromoteToEnvPermission`). Reuse them so the PR path respects the same rules.
- `alpha.tsx` registers the tab (`componentDeployEntityContent` , path `/environments`) and the Overview card (`deploymentStatusCard`) as NFS blueprints. New cards or tabs follow the same `EntityCardBlueprint` and `EntityContentBlueprint` pattern. Hiding or overriding stock extensions is supported.

3.4 Scaffold conventions to follow

- Actions use `createTemplateAction` with zod schemas, registered in `module.ts` with `scaffolderActions.addActions(...)` . Existing actions get `config` , `catalog` , `auth` and `immediateCatalog` .
- Templates that call OpenChoreo must set `EXPERIMENTAL_formDecorators: [{ id: openchoreo:inject-user-token }]` . The decorator (`src/scaffolder/openChoreoTokenDecorator.ts`) injects the user's **OpenChoreo IDP**

token as the `OPENCHOREO_USER_TOKEN` secret. It is a no-op when `openchoreo.features.auth.enabled` is false.

- Form fields to reuse: `NamespaceEntityPicker`, `DeploymentPipelinePicker`, `ComponentNamePicker`, `GitSourceField`, `GitSecretField`.
- Feature flags follow the `openchoreo.features.<name>.enabled` convention in app-config, with matching Helm values (`backstage.features.*`).

4. Required changes by repo

4.1 openchoreo/backstage-plugins (primary)

#	Change	Where	Size	Notes
B1	GitOps scaffolder actions module: <code>openchoreo:gitops:resolve-path</code> (can include validation), <code>openchoreo:gitops:check-conflicts</code> , <code>openchoreo:gitops:render</code>	New package <code>plugins/scaffolder-backend-module-openchoreo-gitops</code> , or new files beside the existing actions. Register like <code>module.ts</code> .	M	<code>render</code> clones the GitOps repo into the workspace, runs <code>occ componentrelease generate</code> and/or <code>occ releasebinding generate --mode file-system --root-dir <workspace></code> , and leaves files for <code>publish:github:pull-request</code> . Fail the task if <code>occ</code> exits non-zero.
B2	Conflict check implementation	Inside <code>check-conflicts</code>	M	Use Backstage's <code>ScmIntegrations</code> and GitHub credentials provider with Octokit to list open PRs on the base branch and compare changed file paths to the resolved target path. Also detect "merged but not yet synced" via the commit status or Flux revision. Return structured output so the template can stop with a clear message.
B3	Promotion template	<code>templates/promote-openchoreo-component/template.yaml</code> (new)	S	Adapt your <code>scaffolder.yaml</code> : add the token decorator, replace the environment enum with the pipeline picker, replace inline path logic with action outputs. Make it deep-linkable (<code>?formData=</code>) so the Deploy tab can prefill it.
B4	Overrides-to-PR path	New <code>openchoreo:gitops:patch-binding</code> action, plus a template	M	No <code>occ</code> support (section 2). Patch <code>componentTypeEnvironmentConfigs</code> , <code>traitEnvironmentConfigs</code> and <code>workloadOverrides</code> in the existing binding file, preserving other fields. Post-MVP if time is short.

#	Change	Where	Size	Notes
B5	GitOps mode switch	plugins/openchoreo-common/src/types/features.ts , app-config, Helm values	S	Add openchoreo.features.gitops.enabled plus the GitOps repo settings (owner, repo, base branch, layout). [Inference] Per-project or per-namespace mode is more correct but needs an annotation on the entity and is a larger change. Start global.
B6	Deploy tab: redirect Promote	useEnvironmentActions.handlePromote, usePromotionAction, PromotePrimaryAction, EnvironmentActions, PipelineCanvas	M	When GitOps mode is on, navigate to the promote template with prefilled namespace, project, component, source and target, instead of calling promoteToEnvironment. Relabel the button (for example "Open promotion PR").
B7	Deploy tab: redirect or block other writes	Same area, plus EnvironmentOverridesPage	M to L	Overrides save becomes "Open PR" (needs B4). Undeploy, redeploy, delete binding, restart, auto-deploy and release creation should be hidden or show an explanation in GitOps mode.
B8	Change tracker backend plugin	New plugins/openchoreo-gitops-backend	L	Stores a record per change: template run, PR number and state, merge SHA, synced revision, binding status. Fed by polling or GitHub webhooks (the portal already receives webhooks on /api/events/http/openchoreo). Exposes a small REST API. This is the research-heavy part.
B9	Status UI	New EntityCardBlueprint (pending changes) and a status chip in the environment card or DeploymentStatusCard	M	Lifecycle: PR Open, In review, Merged, Synced, Ready or Blocked, plus Closed. The "Ready" half already exists (fetchEnvironmentInfo returns binding status). The PR and sync half comes from B8.
B10	Ship occ to the backend	packages/portal-backend build, Dockerfile, Helm chart	M	See 4.2.
B11	Register the template	Helm chart backstage.appConfig.catalog.locations (platform repo) or app-config.yaml	S	Remember Backstage replaces arrays: re-declare the bundled default locations.
B12	Tests	Per package	M	Mock the action context (createMockActionContext), fake occ binary for unit tests, throwaway GitHub repo for integration.

4.2 Decision needed: how `occ` reaches the scaffolder (B10)

Option	Pros	Cons
Bundle the <code>occ</code> binary into the portal image	Simple to call (<code>child_process</code>), same behaviour as the CLI.	Must pin <code>occ</code> to the platform version. Image build change in the platform's Helm and Docker flow.
Run <code>occ</code> in the workflow plane (a Workflow that clones, renders and opens the PR)	Matches how build-and-release GitOps workflows already work. Handles long waits.	Slower for small edits. More moving parts. Needs a new Workflow definition.
Reimplement generation in TypeScript	No binary.	Duplicates <code>occ</code> logic and drifts from it. Not recommended.

[Inference] Start with bundling the binary for the spike and MVP. It is the fastest path to a working demo and keeps `occ` semantics intact. Revisit the workflow-plane route if you want the portal, `occ` and MCP to share one PR path (the idea in your landscape document).

4.3 `openchoreo/openchoreo`

Required for MVP: none found. The controllers already tolerate GitOps-driven changes (for example `internal/controller/releasebinding/controller_watch.go` handles "GitOps sync stamps"). The API's promote endpoint stays unchanged because the portal simply will not call it in GitOps mode.

Optional upstream contributions (discuss on GitHub first, as your proposal already plans):

Change	Where	Why
Pattern-based layout in <code>release-config.yaml</code>	<code>internal/occ/fsmode/config/release_config.go</code> and <code>output/</code>	Only if directory maps are not enough for your configurable-layout deliverable.
Cluster-state fallback for ComponentTypes and Traits	<code>internal/occ/fsmode</code> index and resolver	Your "hybrid dependency resolution". The index already handles repo-resident ClusterComponentTypes and ClusterTraits, so a live-cluster fallback would be a new resolver path. Needs unit tests and mentor agreement.

Change	Where	Why
Override flags for file mode (<code>--set equivalent on releasebinding generate</code>)	<code>internal/occ/cmd/releasebinding</code>	Would remove the need for B4's custom patcher. <code>internal/occ/cmd/setoverride</code> already contains the path-setting engine.
Publish an <code>occ</code> container image or release artifact for embedding	<code>cmd/occ/Dockerfile</code> , release pipeline	Eases B10.
Server-side guard for GitOps-owned resources	<code>internal/openchoreo-api</code> , <code>authz</code>	Stops direct API writes outside the portal. Post-MVP and invasive.

5. Suggested build order

This keeps your "guaranteed MVP" (GitHub + Flux + ReleaseBinding) first.

1. **Spike (week 1 to 2).** In a throwaway repo, run `occ releasebinding generate --mode file-system --root-dir . --project P --component C --target-env staging --use-pipeline <pipeline>` and confirm it updates a binding file. Confirm the portal image story for `occ` . Confirm how `OPENCHOREO_USER_TOKEN` and GitHub credentials coexist in one template run.
2. **B1 (render + resolve-path) and B3 (template).** Get a PR opening from the Create page with no UI change.
3. **B2 (conflict gate).** Add the stop-and-explain behaviour and the concurrent-promotion acceptance test.
4. **B5 and B6.** Add the flag and redirect the Promote button.
5. **B8 and B9.** Change tracker and status chip. Most research value, most risk.
6. **B4 and B7.** Overrides via PR and blocking of other writes.
7. Flux 2.8 commit-status and PR-comment providers can feed B8 with less polling, if you want to avoid writing a Flux plugin.

6. Decisions to take with your mentor

- **PR authorship.** The user token in the template is the OpenChoreo IDP token, not a GitHub token. `publish:github:pull-request` will use the Backstage GitHub integration credentials (a bot or app) unless you add the GitHub sign-in provider (already present in `portal-backend`) and pass the user's GitHub token. This decides audit attribution and CODEOWNERS behaviour.
- **Where the mode flag lives** (global, per namespace, per project).

- **Whether layout needs patterns** or the existing `release-config.yaml` is enough.
- **How `occ` is packaged** (4.2).

7. What I did not verify

- I did not build the plugins, run the portal, or execute `occ`. Command behaviour is from reading source and tests, so run the spike in step 1 to confirm.
- I did not inspect the platform Helm chart (`install/helm`), so how `catalog.locations` and the image are wired is unconfirmed beyond the docs.
- The Deploy tab's visual layout is inferred from component names.
- The docs describe build-and-release and bulk-promote workflows that open PRs, but I found no such workflow definitions in the main platform repo's `samples` or `install` folders, so I could not read them. They may live in a separate GitOps examples location.